

CS & IT ENGINEERING



Data Structure & Programming

Practice Session

Lecture No.- 01



By- Satya sir

Topics to be Covered



Practice Session





Topic : PYQ Practice - 1



↑ Random access

#Q. A queue is implemented using an array such that ENQUEUE and DEQUEUE operations are performed efficiently. Which one of the following statements is CORRECT (n refers to the number of items in the queue)?

Enqueue()
 $x = x + 1$

Dequeue()
 $\text{Print } Q[\text{front}]$
 $\text{front} = \text{front} + 1$

- ✓ A. Both operations can be performed in $O(1)$ time. $Q[x] = \text{value}$
- B. At most one operation can be performed in $O(1)$ time but the worst case time for the operation will be $O(n)$.
- C. The worst case time complexity for both operations will be $O(n)$.
- D. Worst case time complexity for both operations will be $O(\log n)$

GATE 2016



Topic : PYQ Practice - 1



#Q. Consider the following statements:

- ~~(i)~~ First-in-first out types of computations are efficiently supported by STACKS.
- ~~(ii)~~ Implementing LISTS on linked lists is more efficient than implementing LISTS on an array for almost all the basic LIST operations. *[Insertion, Deletion operations; array - Contiguous memory, LL - Random memory]*
- ~~(iii)~~ Implementing QUEUES on a circular array is more efficient than implementing QUEUES on a linear array with two indices.
- ~~(iv)~~ Last-in-first-out type of computations are efficiently supported by QUEUES.

Which of the above statements are TRUE?

~~A)~~ ii and iii

~~B)~~ i and iii

~~C)~~ iii and iv

~~D)~~ ii and iv

GATE 1996

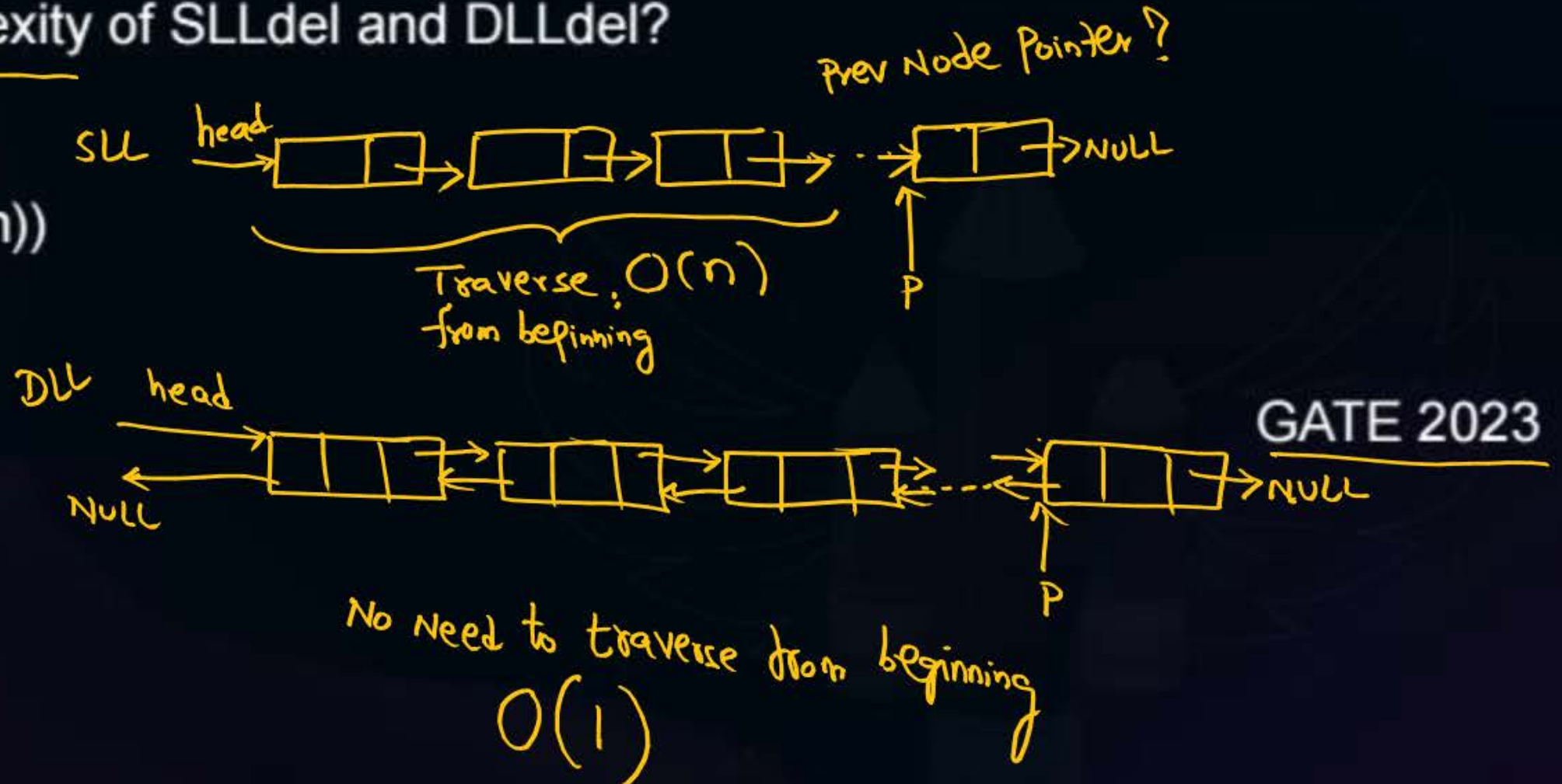


Topic : PYQ Practice - 1



#Q. Let SLLdel be a function that deletes a node in a singly-linked list given a pointer to the node and a pointer to the head of the list. Similarly, let DLLdel be another function that deletes a node in a doubly-linked list given a pointer to the node and a pointer to the head of the list. Let n denote the number of nodes in each of the linked lists. Which one of the following choices is TRUE about the worst-case time complexity of SLLdel and DLLdel?

- A. SLLdel is $O(1)$ and DLLdel is $O(n)$
- B. Both SLLdel and DLLdel are $O(\log(n))$
- C. Both SLLdel and DLLdel are $O(1)$
- D. ✓ SLLdel is $O(n)$ and DLLdel is $O(1)$





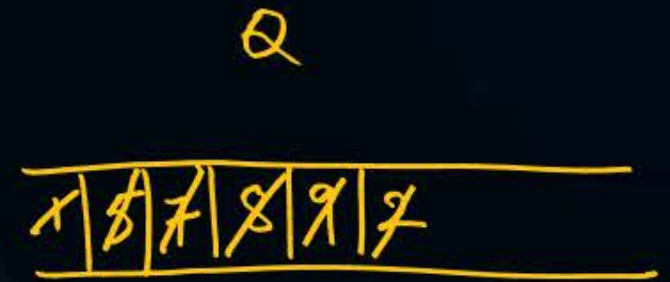
Topic : PYQ Practice - 1



#Q. Consider a sequence a of elements $a_0 = 1$, $a_1 = 5$, $a_2 = 7$, $a_3 = 8$, $a_4 = 9$, and $a_5 = 2$. The following operations are performed on a stack S and a queue Q , both of which are initially empty.

- I: push the elements of a from a_0 to a_5 in that order into S . ✓
- II: enqueue the elements of a from a_0 to a_5 in that order into Q . ✓
- III: pop an element from S . ✓
- IV: dequeue an element from Q . ✓
- V: pop an element from S . ✓
- VI: dequeue an element from Q . ✓
- VII: dequeue an element from Q and push the same element into S .
- VIII: Repeat operation VII three times.
- IX: pop an element from S .
- X: pop an element from S .

The top element of S after executing the above operations is 8.





Topic : PYQ Practice - 1



#Q. A function f defined on stacks of integers satisfies the following properties. $f(\emptyset) = 0$ and $f(\text{push}(S, i)) = \max(f(S), 0) + i$ for all stacks S and integers i . If a stack S contains the integers 2, -3, 2, -1, 2 in order from bottom to top, what is $f(S)$?

$$f(\text{push}(S, 2)) = \max(0, 0) + 2 = 0 + 2 = 2$$

$$f(\text{push}(S, -3)) = \max(2, 0) + (-3) = 2 + (-3) = -1$$

$$f(\text{push}(S, 2)) = \max(-1, 0) + 2 = 0 + 2 = 2$$

$$f(\text{push}(S, -1)) = \max(2, 0) + (-1) = 2 + (-1) = 1$$

$$f(\text{push}(S, 2)) = \max(1, 0) + 2 = 1 + 2 = 3$$

$$f(\) \quad \boxed{\cancel{\emptyset} \cancel{2} \cancel{-1} \cancel{2} \cancel{-3} \quad 3}$$

Ans: 3

GATE 2005

2
-1
2
-3
2



Topic : PYQ Practice - 1

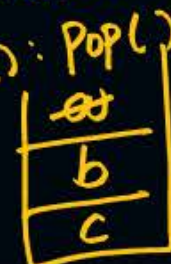


#Q. Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions. Which one of the following statements is TRUE with respect to this modified stack?

A) A queue cannot be implemented using this stack.



Dequeuel(): Reverse, Pop $\Rightarrow$ a
Dequeuel(): Pop()



(B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions.

Reverse, Push, Reverse

☒ (C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction. *Pop*

(D) A queue can be implemented where both ENQUEUE and DEQUEUE take a single instruction each.

GATE 2014

Let Enqueue a, b, c

Dequeue $\Rightarrow$ a, b, c

Enqueue(a)

a

① Reverse Stack

② Push a

③ Reverse Stack

Enqueue(b)

b
a

 $\Rightarrow$

a
b

1) Reverse Stack

2) Push b

3) Reverse Stack

Enqueue(c)

1) Reverse Stack

b
a

2) Push c

c
b
a

3) Reverse Stack

a
b
c

Dequeue() $\Rightarrow$ Pop(Stack) $\Rightarrow$ a

Dequeue() $\Rightarrow$ Pop(Stack) $\Rightarrow$ b

Dequeue() $\Rightarrow$ Pop(Stack) $\Rightarrow$ c



Topic : PYQ Practice – 1 – GATE 2025

#Q. Let S1 and S2 be two stacks. S1 has capacity of 4 elements. S2 has capacity of 2 elements S1 already has 4 elements: 100, 200, 300, and 400, whereas S2 is empty, as shown below.

Only the following three operations are available:

PushToS2: Pop the top element from S1 and push it on S2.

PushToS1: Pop the top element from S2 and push it on S1.

GenerateOutput: Pop the top element from S1 and output it to the user.

Stack S2

Stack S1
400 (Top)
300
200
100

Note that the pop operation is not allowed on an empty stack and the push operation is not allowed on a full stack.

Which of the following output sequences can be generated by using the above operations? **[MSQ]**

- ☒ A) 100 200 300 400 ☒ B) 200 300 400 100 ☒ C) 400 200 100 300 ☒ D) 300 200 400 100

Handwritten notes for option B:
Push(S2), Push(S2), GO(), Push(S1), GO, Push(S1), GO, GO
S2: [200], [300]
S1: [400], [300], [200], [100]

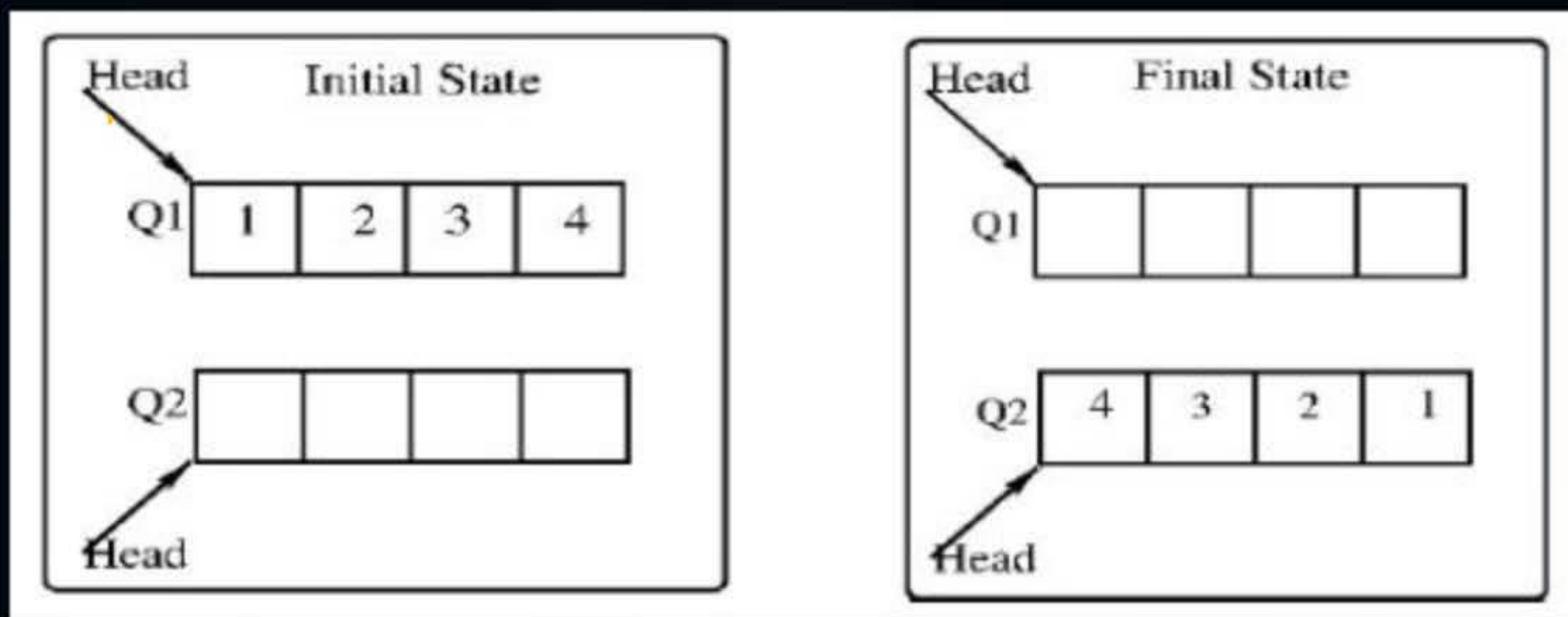
Handwritten notes for option C:
GO, Push(S2), GO, GO, Push(S1), GO, Push(S2), GO, GO, Push(S1), GO, GO
S2: [300], [400]
S1: [400], [200], [100], [300]



Topic : PYQ Practice - 1



#Q. Consider the queues Q1 containing four elements and Q2 containing none (shown as the Initial State in the figure). The only operations allowed on these two queues are Enqueue (Q, element) and Dequeue (Q). The minimum number of Enqueue operations on Q1 required to place the elements of Q1 in Q2 in reverse order (shown as the Final State in the figure) without using any additional storage is 0.



1st time

$\text{enqueue}(q_2, \text{dequeue}(q_1))$
 $q_2 [1] [2|3|4] q_1$
 $\text{enqueue}(q_2, \text{dequeue}(q_2))$
 $q_2 [1] [2|3|4] q_1$

GATE 2022

2nd time

$q_2 [1|2] q_1 [3|4]$
 $q_2 [2|1] q_1 [3|4]$

3rd

$q_2 [2|1|3] q_1 [4]$
 $q_2 [1|3|2] q_1 [4]$
 $q_2 [3|2|1] q_1 [4]$



Let LIST be a datatype for an implementation of linked list defined as follows:

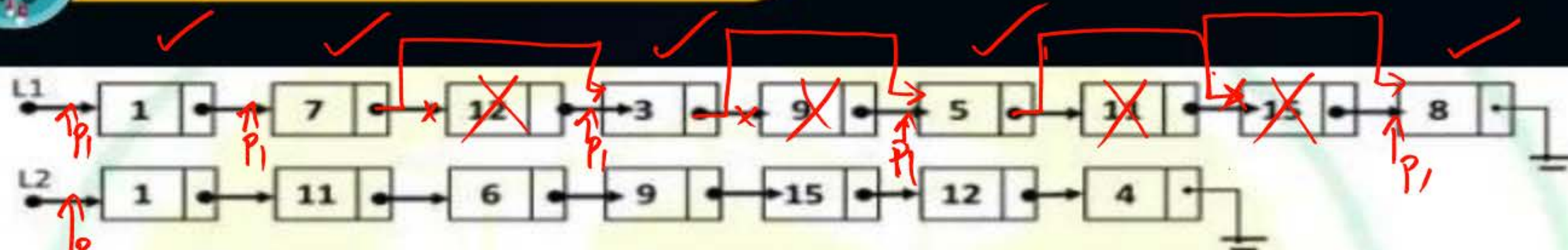
```
typedef struct list {  
    int data;  
    struct list *next;  
} LIST;
```

Suppose a program has created two linked lists, L1 and L2, whose contents are given in the figure below (code for creating L1 and L2 is not provided here). L1 contains 9 nodes, and L2 contains 7 nodes.

Consider the following C program segment that modifies the list L1. The number of nodes after the execution of the code segment is _____. (Answer in integer)



Topic : PYQ Practice - 1



```
int find (int query, LIST *list) {
    while (list != NULL) {
        if(list->data == query) return 1;
        list = list->next;
    }
    return 0;
}

int main () {
    ... ..
    ptr1=L1; ptr2=L2;
    while (ptr1->next != NULL) {
        query = ptr1->next->data;
        if (find (query, L2))
            ptr1->next = ptr1->next->next;
        else ptr1 = ptr1->next;
    }
    ... ..
    return 0;
}
```

Handwritten notes: 7, 9, 11, 15 (written in red) and a red arrow pointing to the main function.



H/W

#Q. Suppose you are given an implementation of a queue of integers. The operations that can be performed on the queue are:

- i. isEmpty (Q) — returns true if the queue is empty, false otherwise.
- ii. delete (Q) — deletes the element at the front of the queue and returns its value.
- iii. insert (Q, i) — inserts the integer i at the rear of the queue.

Consider the following function:

```
void f (queue Q) {  
    int i ;  
    if (!isEmpty(Q)) {  
        i = delete(Q);  
        f(Q);  
        insert(Q, i);  
    }  
}
```




What operation is performed by the above function f ?

- (a) Leaves the queue Q unchanged
- (b) Reverses the order of the elements in the queue Q
- (c) Deletes the element at the front of the queue Q and inserts it at the rear keeping the other elements in the same order
- (d) Empties the queue Q



An implementation of a queue Q, using two stacks S1 and S2 is given below

```
void insert (Q, x) {  
    push (S1, x);  
}  
void delete (Q) {  
    if (stack-empty (S2)) then  
        if (stack-empty (S1)) then {  
            print ("Q is empty");  
            return;  
        }  
        else while (! (stack-empty (S1))) {  
            x = pop (S1);  
            push (S2, x);  
        }  
    x = pop (S2);  
}
```

Let n insert and $m(\leq n)$ delete operations be performed in an arbitrary order on an empty queue Q. Let x and y be the number of push and pop operations performed respectively in the process. Which one of the following is true for all m and n ?



Topic : PYQ Practice - 1



- A. $n + m \leq x < 2n$ and $2m \leq y \leq n + m$
- B. $n + m \leq x \leq 2n$ and $2m \leq y \leq 2n$
- C. $2m \leq x \leq 2n$ and $2m \leq y \leq n + m$
- D. $2m \leq x < 2n$ and $2m \leq y \leq 2n$

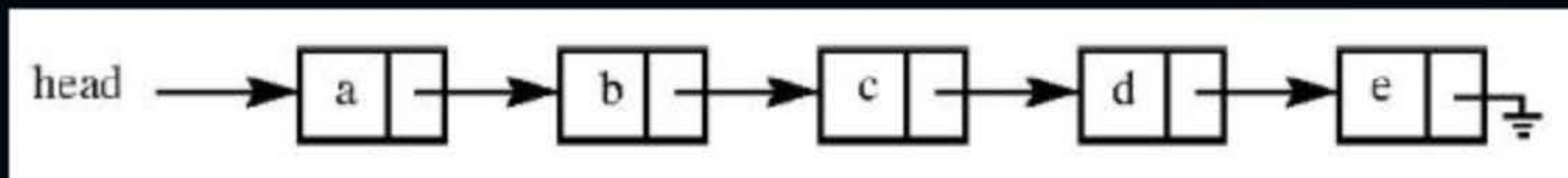


Topic : PYQ Practice - 1

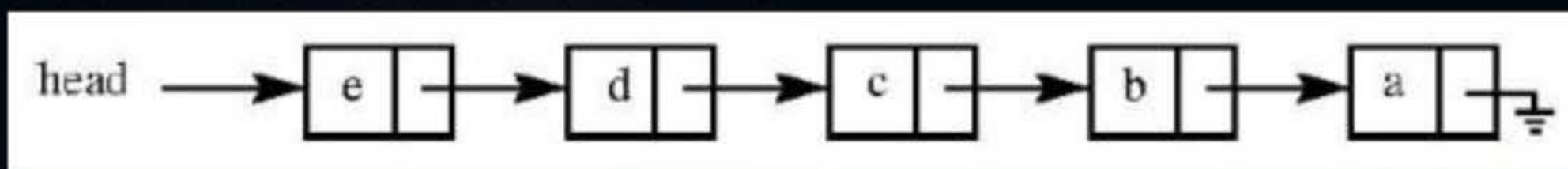
H/W



#Q. Consider the problem of reversing a singly linked list. To take an example, given the linked list below,



The reversed linked list should look like



Which one of the following statements is TRUE about the time complexity of algorithms that solve the above problem in $O(1)$ space?

- A. The best algorithm for the problem takes $\Theta(n)$ time in the worst case.
- B. The best algorithm for the problem takes $\Theta(n \log n)$ time in the worst case.
- C. The best algorithm for the problem takes $\Theta(n^2)$ time in the worst case.
- D. It is not possible to reverse a singly linked list in $O(1)$ space.



The minimum number of comparisons required to determine if an integer appears more than $n/2$ times in a sorted array of n integers is

- A. $\Theta(n)$
- B. $\Theta(\log n)$
- C. $\Theta(n * \log n)$
- D. $\Theta(1)$

GATE 2008

THANK - YOU